

EXAMEN SEGUNDO PARCIAL PMDM

La prueba consiste en aplicar la arquitectura **Clean Architecture** al desarrollo de una aplicación Android que persiste los datos en una base de datos local mediante Room.

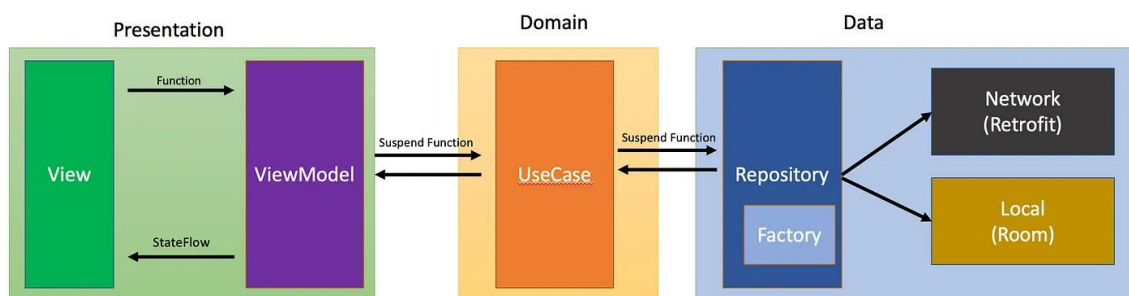
La entrega de la app se debe realizar en la plataforma Aules, mediante la subida del proyecto Android Studio comprimido en zip.

La aplicación consiste en un listado de citas acerca del mundo de la informática, con su texto y su autor. Cada cita del listado se puede marcar como favorita o verla en una vista de detalle, donde se podrá, además de añadirla a favoritas, compartirla. La propia pantalla de listado permite las operaciones de filtrado de citas por texto (cita o autor), ver solamente las citas favoritas y ver una cita aleatoria en la vista de detalle.

La aplicación es funcional tal como se entrega (podéis probarla en ejecución), pero no persiste la información y no aplica ninguna arquitectura escalable y que permita mantener el estado de la UI cuando se producen cambios de configuración en el dispositivo.

Tu labor consiste en aplicar los conocimientos adquiridos durante la segunda parte del curso para proporcionar a la aplicación la funcionalidad que permita persistir la información y mantener el estado de la UI cuando cambie la configuración del dispositivo (por ejemplo, una rotación) y todo ello aplicando las buenas prácticas de diseño definidas en la arquitectura Clean:

- Separación de responsabilidades en tres capas:
 - **Data:** Persistencia de la información
 - **Domain:** Lógica de la aplicación
 - **Presentation:** Interacción con el usuario y mostrar información
- Domain es una capa que no depende de ninguna otra (es el núcleo central) ni depende de ningún artefacto de Android (es código Kotlin puro)
- Data y Presentation son capas de infraestructura y si son dependientes de Android y de Dominio, pero no se conocen entre si



Instrucciones iniciales

Descarga de aules el fichero zip con la plantilla inicial del proyecto y el pdf con el gui3n.

Descomprime el archivo zip en una carpeta y 3brela con Android Studio.

En la plantilla ver3s varias clases en los paquetes data, domain y ui.

El paquete data contiene una clase **QuoteProvider**, este provider proporciona una lista de citas (objetos de tipo **Quote**) almacenadas en memoria.

En el paquete **domain** se dispone de:

- La clase **QuoteManager**, que mantiene una lista observable por Compose de citas y se encarga de realizar las operaciones de filtrado, obtener una cita por su id y de establecerla como favorita o no.
- La clase **Quote** que es el modelo de datos del dominio.

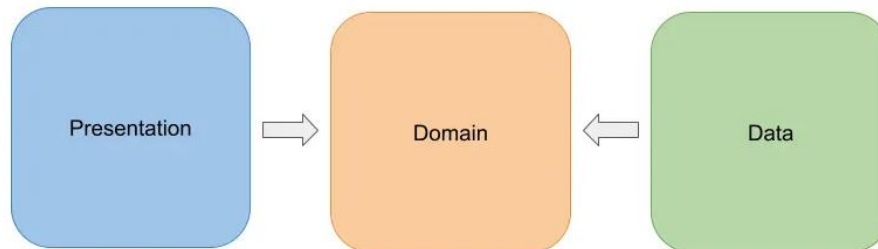
Dentro del paquete ui tenemos varias cosas:

- Los componentes, elementos composables para a3adir en las pantallas de la App.
- Pantallas, (tres pantallas):
 - AboutScreen
 - QuoteDetailsScreen
 - QuoteListScreen
- Algunas funciones de utilidad
 - para obtener el color y las iniciales del avatar del autor de la cita
 - para compartir una cita
- Constantes
- El composable ra3z de la navegaci3n entre pantallas NavigationRoot

Al finalizar la adaptaci3n de la App la clase QuoteManager se podr3 eliminar completamente y los datos se persistir3n en una base de datos local, de manera que no se perder3 la informaci3n al salir de la aplicaci3n.

Capa de dominio

La capa de dominio en Clean Architecture es la capa nuclear, no depende de ninguna otra capa (en otras palabras, no conoce ninguna clase o código de las otras dos capas).

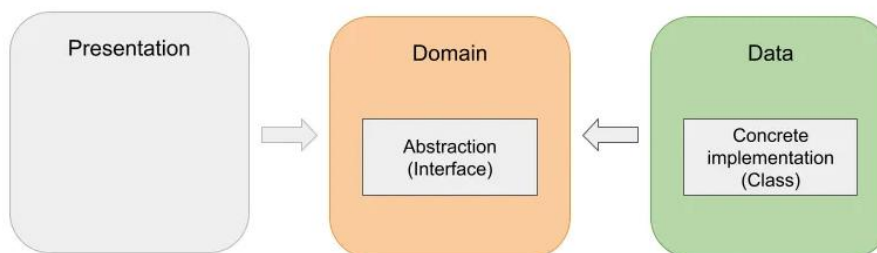


Modelos de dominio

En la capa de dominio se definen los modelos de datos de dominio. En este caso la clase **Quote** que ya la tienes disponible

Repositorios

También se define la interface de repositorio que actúa como contrato de la funcionalidad que debe implementar la clase implementadora del repositorio que se defina en la capa de datos. Se aplica el principio de inversión de dependencias para que Data dependa de Domain.



Define la interface de repositorio para las citas **QuoteRepository** en el subpaquete “repository” dentro del paquete “domain”.

Los métodos de la interface repositorio serán los siguientes:

getQuotes() → Devuelve un flow que emite una lista de citas (modelo del dominio)

toggleFavorite(quote:Quote) → alterna el valor de favorita de la cita

searchQuotes(query: String) → Devuelve un flow que emite una lista de citas filtradas

getQuoteById(id: Int) → Devuelve un flow que emite la cita que corresponde con el id

getRandomQuote() → Devuelve un flow que emite una cita aleatoria

Recuerda que toda función que no devuelva un Flow debe ser declarada como una función suspendida.

Casos de uso

Tendrás que crear en el paquete dominio un subpaquete “usecase” para albergar todas las clases que modelan los casos de uso de la app.

ToggleFavoriteUseCase

Este caso de uso permite al usuario alternar entre que una determinada cita sea considerada favorita o no. Debe delegar el cambio de valor de favorito de la cita en un objeto **QuoteRepository** del que dependerá.

GetRandomQuoteUseCase

Este caso de uso permite obtener los datos de una cita aleatoria de entre todas las citas que están almacenadas en el sistema de persistencia. Para obtenerla hará uso de los servicios que le presta un **QuoteRepository** del que por tanto dependerá.

GetFilteredQuotesUseCase

El cometido de este caso de uso es obtener a partir de un texto utilizado para filtrar citas una lista de citas filtradas, es decir las citas cuyo texto o autor contiene el texto proporcionado para filtrar. Simplemente delega en un **QuoteRepository**.

Al invocarlo hay que proporcionarle la query que contiene el texto para filtrar las citas.

GetQuoteUseCase

Este caso de uso permite obtener la cita cuyo id se corresponde con el identificador proporcionado para su identificación. Nuevamente delega en un **QuoteRepository**.

Necesita que se le proporcione el identificador de la cita a recuperar del almacén de datos.

Los casos de uso se deben crear como clases que contienen un único método público que sea el operador **invoke**. De esta manera se facilita la llamada al caso de uso puesto que si por ejemplo, tenemos una referencia al caso de uso `GetRandomQuoteUseCase` en la variable `getRandomQuoteUseCase`, podemos llamar al método `invoke` de dos maneras:

- `getRandomQuoteUseCase()` → forma sencilla e implícita
- `getRandomQuoteUseCase.invoke()`

Recuerda:

- El método `invoke` debe ser declarado como suspendido a no ser que devuelva un `Flow`.

Capa de datos

En el paquete “data” colocamos todo el código cuya responsabilidad de la persistencia de la información que maneja la aplicación. En este caso vamos a persistir las citas en una base de datos local SQLite mediante el ORM Room. Como la base de datos es local crea un subpaquete “local” dentro de “data”.

Room es un ORM, y trabaja con modelos de datos de la capa de datos para persistir en un base de datos relacional, más conocidos como entidades.

Entidad

Crea la entidad QuoteEntity en el subpaquete “entity” y mapéala mediante anotaciones para que se guarden sus instancias como filas en una tabla con nombre “quotes”.

La clave primaria será autogenerada

Debe contar con los atributos: ID, TEXT, AUTHOR, IS_FAVORITE

Dao

El DAO es el Data Access Object y su responsabilidad es transformar llamadas a métodos en la ejecución de una consulta SQL que realiza una acción en la base datos que puede ser de inserción, actualización, borrado o selección: CRUD

Crea en la carpeta “local” un QuoteDao que realice las siguientes operaciones:

- **getAllQuotes()** → Devuelve un Flow que emite todas las citas de la BD
- **getQuoteById(id:Int)** → Devuelve un Flow que emite la cita con el id proporcionado
- **deleteAllQuotes()** → Borra todas las citas de la BD
- **getQuotesCount()** → Devuelve el número de total de citas en la BD
- **searchQuotes(query:String)** → Devuelve un flow que emite todas las citas que pasan el filtro indicado por query, todas las citas que contengan el texto del parámetro query en el texto de la cita o en el nombre del autor

PISTA: usa esta consulta

```
SELECT * FROM quotes WHERE text LIKE '%' || :query || '%' OR author LIKE '%' || :query || '%'
```

- **getRandomQuote()** → Devuelve un Flow que emite una cita aleatoria

PISTA: usa esta consulta

```
SELECT * FROM quotes ORDER BY RANDOM() LIMIT 1
```

- **insertQuote** e **insertQuotes** como métodos para insertar una cita y una lista de citas
- **updateQuote** y **updateQuotes** para actualizar una cita o una lista

- **toggleFavorite**(quoteId: Int) → alterna el valor del campo is_favorite en la cita con id igual al proporcionado por el parámetro quoteId

PISTA: usa esta consulta

UPDATE quotes SET is_favorite = NOT is_favorite WHERE id = :quoteId

RECUERDA:

- Las funciones que devuelven flows pueden ser funciones normales
- Las funciones que modificación de datos y las que no devuelven flows deben ser funciones suspendidas

Base de datos

Crea la clase AppDatabase en la carpeta “local” para que maneje una base de datos con un esquema que contenga una tabla para la entidad QuoteEntity. La clase debe declarar un método quoteDao que devuelva una referencia a un objeto de tipo QuoteDao para realizar operaciones con las citas de la base de datos.

Usa las anotaciones necesarias.

Para iniciar una instancia de base de datos se necesita una referencia al contexto de la aplicación. Por eso vamos a definir estos dos métodos de extensión de Context dentro del fichero donde se declara la clase AppDatabase

El método **getDatabase** devuelve una base de datos que persiste la información de manera permanente ya que usa un fichero como almacén de la base de datos.

```
fun Context.getDatabase(): AppDatabase {
    return Room.databaseBuilder(
        this,
        AppDatabase::class.java,
        "quotes.db"
    ).build()
}
```

La función **getInMemoryDatabase** devuelve una base de datos que trabaja directamente en memoria y por tanto, las acciones no son persistentes.

```
fun Context.getInMemoryDatabase(): AppDatabase {
    return Room.inMemoryDatabaseBuilder(
        this,
        AppDatabase::class.java
    ).build()
}
```

Podrás elegir una u otra cuando declares la dependencia en el módulo de Koin.

Mappers

En la capa de datos se implementa la interface del repositorio cuyos métodos trabajan con referencias de tipo Quote (modelo del dominio) y no QuoteEntity que es una modelo de datos (y que no debe ser expuesto al resto de capas). Por tanto, debemos convertir un QuoteEntity a un Quote equivalente y viceversa. Esta función la realizan los mappers, funciones que convierten un objeto del modelo de datos en un objeto del modelo de dominio y a la inversa.

Para convertir un QuoteEntity en un Quote define una función de extensión de QuoteEntity y llámala toDomain()

Para convertir un Quote en un QuoteEntity define una función de extensión de Quote y llámala toDatabase()

Define estas funciones en un archivo QuoteMappers.kt en la carpeta “local” de “data”.

Repositorio

En la capa de datos se implementa la interface de repositorio declarada en la capa de dominio.

Crea una subcarpeta “repository” dentro de data y crea dentro una clase **QuoteRepositoryImpl** que implemente la interface **QuoteRepository**.

La clase QuoteRepositoryImpl necesita una referencia a un QuoteDao para poder llevar a cabo su trabajo, que básicamente consiste en delegar en el QuoteDao las acciones y mapear los resultados que este le entrega, ya que el QuoteDao utiliza QuoteEntity.

Por tanto, apóyate en los métodos mapeadores **toDomain** y **toDatabase** para transformar tanto listas como objetos QuoteEntity a Quote y viceversa cuando sea necesario.

Capa de presentación

Reubicación de elementos de la UI en Compose

Crea un paquete “presentation” para la capa de presentación, dentro una subcarpeta “screens” y dentro de esta última una subcarpeta por cada una de las 3 pantallas.

Mueve los archivos que contienen cada una de las pantallas a su correspondiente subcarpeta y crea dentro un paquete “components” para los componentes específicos de esa pantalla.

También tendrás que crear un paquete “components” directamente dentro de “presentation” para albergar los componentes composables no específicos de una pantalla puesto que se reutilizan en más de una pantalla.

Mueve también el paquete utils desde ui a presentation.

ViewModels

Crea un viewModel para la pantalla de **QuoteDetails** y otro para la pantalla de **QuoteList**, llámalos igual que la pantalla y añade el sufijo ViewModel.

Como se va a seguir el **patrón MVI**, define la interface que contiene las acciones que puede realizar el usuario cuando interactúa con la pantalla.

En el viewModel declara un método **onAction** que recibe una referencia a la acción efectuada por el usuario, comprueba el tipo concreto de la acción y llama a una función privada creada para manejarla.

Para el caso de la pantalla de detalle las acciones son las siguientes (QuoteDetailsAction):

- **OnFavoriteClick** (el usuario ha tocado el chip de favorito)
- **OnShareClick** (el usuario ha tocado el chip de compartir)

Para el caso de la pantalla de lista de citas las acciones son las siguientes (QuoteListAction):

- **OnRandomQuote** (el usuario ha tocado el botón del dado para ver una cita aleatoria)
- **OnAbout** (para navegar a la pantalla de Acerca de)
- **OnToggleFilterFavorites** (el usuario ha tocado el icono favoritos de la topbar)
- **OnToggleSearchBar** (el usuario ha tocado el icono lupa que muestra u oculta la barra de búsqueda)
- **OnToggleFavorite** (el usuario ha tocado el icono favorito de una cita de lista)
- **OnQueryChange** (el usuario ha modificado la caja de texto de búsqueda)
- **OnQuoteSelected** (el usuario ha seleccionado al tocar una cita de la lista)

Eventos MVI

Pega el siguiente código en un archivo `ObserveAsEvents.kt` dentro del paquete `presentation` y usalo para procesar los eventos.

```
@Composable
fun <T> Flow<T>.ObserveAsEvents(onEvent: suspend (T) -> Unit) {

    val lifecycleOwner = LocalLifecycleOwner.current

    LaunchedEffect(lifecycleOwner) {

        lifecycleOwner.lifecycle.repeatOnLifecycle(Lifecycle.State.STARTED) {
            withContext(Dispatchers.Main.immediate) {
                collect { event -> onEvent(event) }
            }
        }
    }
}
```

Para la pantalla de lista de citas

Cuando el usuario pulsa en el icono del dado se ha de navegar a la pantalla de detalle para mostrar una cita aleatoria. Es decir, en respuesta a la acción `OnRandomQuote` hay que obtener una cita aleatoria y navegar a la pantalla de detalle para mostrarla.

Maneja esto mediante un evento de un solo uso `OnNavigateToQueryDetails`.

Define este evento como una implementación de la interface `QuoteListEvent`.

Crea un `Channel< QuoteListEvent>` en el `viewModel` para la lista de citas.

Cuando manejes la acción `OnRandomQuote` obtén la cita aleatoria mediante el caso de uso `GetRandomQuoteUseCase` y a continuación envía el evento por el canal para que pueda ser observado en la función `QuoteListRoute` usando la función `ObserveAsEvents`.

Para la pantalla de detalle de cita

Desde esta pantalla se puede compartir la cita. Para compartir la cita dispones del método de extensión **share** definido en el archivo `ShareUtils.kt`. Este método necesita que se le proporcione una referencia al contexto y por tanto, no se puede utilizar dentro de una función del `viewModel`.

Por tanto, el `viewModel` debe enviar un evento a la UI para que esta lo observe (recolecte) y desde ahí llamar a `share` pasándole el contexto que podemos obtener en `compose` mediante `LocalContext.current`.

Define un interface para los eventos de UI de la pantalla de detalle y llámalo `QuoteDetailsEvent`.

Define una clase de evento `OnShare` que recibe un parámetro de tipo `Quote` e implementa `QuoteDetailsEvent`.

En el `viewModel` para la vista de detalle crea un canal para enviar eventos de tipo `QuoteDetailsEvent`.

Envía un evento `OnShare` por el canal como respuesta a la acción `OnShare` recibida y generada por el usuario al tocar el botón chip de compartir de la pantalla.

Estado de la UI

El estado de la UI se mantiene en los `viewModels` mediante `StateFlows`.

Para la pantalla de lista de citas crea una clase `QuoteListUiState` que será el tipo de elementos del `StateFlow` y contendrá todos los elementos necesarios para determinar el estado de la pantalla de lista de cita:

- **query** → el texto de la barra de búsqueda (para filtrar citas)
- **quotes** → una lista de citas (filtrada)
- **showSearchBar** → indica si se muestra o no la barra de búsqueda
- **showFilterFavorites** → indica si se muestran todas las citas o solamente las citas favoritas

Para la pantalla de detalle, dado que el estado consiste simplemente en la cita a mostrar no hace falta que crees una clase `QuoteDetailsUiState`, puedes usar directamente un `StateFlow` que emita un `Quote` directamente.

El `viewModel` de la pantalla de detalles se puede apoyar en el caso de uso `GetQuoteUseCase` que devuelve un `Flow<Quote>` y convertirlo en un `StateFlow` mediante la función **`stateIn`**

Filtrado de citas

Para filtrar las citas debes observar (recolectar) el `StateFlow` en el propio `viewModel` para detectar cuando se emite `QuoteListUiState` que tenga un valor del atributo **query** distinto al de la anterior emisión y a continuación usarlo para pedirle al caso de uso `GetFilteredQuotesUseCase` (dependencia) que recupere las citas que pasar le filtro indicado por el valor de `query`. Este devolverá un `Flow<List<Quotes>>` por lo que tenemos que aplicar la función `flatMapLatest` para que el flow de flows `Flow<Flow<List<Quotes>>` se aplane a un `Flow<List<Quotes>>` y ahora por cada elemento (emisión) actualizar el `uiState`, concretamente el atributo `quotes`.

Recuerda:

Para actualizar un `MutableStateFlow` (generar una nueva emisión) se utilizr preferiblemente la función **`update`**.

Por ejemplo, para actualizar el valor de la propiedad `query` de un `QuoteListUiState` referenciado por la variable `_uiState`:

```
_uiState.update { it.copy(query = action.query) }
```

Composables `_Route`

Por cada pantalla a la que se puede navegar y que use un `viewModel` debes crear una función composable con nombre de la pantalla y sufijo `Route` que reciba el `viewModel` y:

- Convierta el estado de la UI en un State observable de Compose
- Procese los eventos
- Llame a la función Composable que muestra la pantalla

Las funciones `_Route` reciben una referencia al `viewModel`.

Además, pueden declarar otros parámetros para recibir:

- Callbacks para navegación. Por ejemplo, un `onBack` para ir a la pantalla anterior, `onAbout` para ir a la pantalla de About, `onNavigateToDetails` para ir a la pantalla de detalles, ...
- El ID de una cita para identificar la cita que se quiere mostrar (en el caso de la pantalla de detalle)

Composable QuoteDetailsRoute

La función para la pantalla de detalles de cita **QuoteDetailsRoute** debería observar los eventos de un solo uso **QuoteDetailsEvent**, en concreto el **OnShare** para cada vez que se envíe un evento de este tipo compartir la cita.

Debe recibir además del `viewModel`:

- **quoteId** → El identificador de la cuota
- **onBack** → callback para la navegación a la pantalla anterior en la pila de `backStack`

Cuando se llama a `QuoteDetailsRoute` desde la función `NavigationRoot` se proporcionan los argumentos para estos parámetros.

Llama a la función composable de la pantalla de detalle de la cita.

Composable QuoteListRoute

La función para la pantalla de lista de citas **QuoteListRoute** tiene que observar los eventos de un solo uso **QuoteListEvent**, en concreto el **OnNavigateToQuoteDetails** para navegar a la pantalla de detalles cuando se observe un evento de este tipo.

Debe recibir, además del `viewModel`, dos parámetros con los callbacks para navegar a la pantalla de About y a la pantalla de detalles:

- **onNavigateToQuoteDetails**: `(Int) -> Unit`
- **onNavigateToAbout**: `() -> Unit`

Cuando se llama a `QuoteListRoute` desde la función `NavigationRoot` se proporcionan los argumentos para estos dos parámetros indicando que claves apilar en la pila de `backStack` para navegar a dichas pantallas cuando sea necesario.

También debería declarar una función (o una variable de tipo función) que reciba una referencia a un `QuoteListAction` y si se trata de un:

- **OnAbout** → que invocará a `onNavigateToAbout`
- **OnQuoteSelected** → que invocará a `onNavigateToQuoteDetails` proporcionando el identificador de la cita
- El resto de las acciones las delega llamando al método `onAction` del `viewModel` pasándole la acción.

Convierte el `StateFlow` del estado de la UI proporcionado por el `viewModel` en un estado de `Compose` mediante la función `collectAsStateWithLifecycle()`

Llama a la función composible de la pantalla de lista de citas.

Composables de pantalla MVI

Son las funciones composibles que definen la UI de una pantalla, no reciben el `viewModel` como parámetro, sino las que tienen sufijo `Route`.

Estas funciones reciben:

- `QuoteListScreen`
 - o `uiState` de tipo `QuoteListUiState`
 - o `onAction` de tipo función que recibe un `QuoteListAction` y no devuelve nada.
- `QuoteDetailsScreen`
 - o `quote` de tipo `Quote?`
 - o `onAction` de tipo función que recibe un `QuoteDetailsAction` y no devuelve nada.
 - o `onBack` de tipo función que no recibe nada y no devuelve nada.

Navegación entre pantallas

La navegación la vamos a situar en la subcarpeta “navigation”

Mueve a esta carpeta el archivo `NavigatioRoot.kt`.

Modifica las entradas del DSL del `entryProvider` para que el contenido asociado con las `NavKeys` llame a las funciones composibles `QuoteDetailsRoute` en lugar de `QuoteDetailsScreen` y `QuoteListRoute` en lugar de `QuoteListScreen`.

Inyección de dependencias

Vamos a utilizar el patrón DI (dependency injection) en el desarrollo de la app. Concretamente usaremos Koin como inyector, para lo que añadimos la dependencia “io.insert-koin:koin-androidx-compose” en el `build.gradle` del módulo.

Habría que añadir una dependencia de tipo `viewModel` por cada `viewModel` y usar la función **`koinViewModel`** en aquellas funciones donde se quiera inyectar como parámetro de entrada.

En la declaración de la dependencia del `viewModel` para la pantalla de detalles hay que tener en cuenta que el `quoteId` se inyecta como parámetro dinámico (dado que su valor se recibe desde el key de navegación)

Para que Koin realice su trabajo en tiempo de ejecución debes iniciar Koin desde el método `onCreate` de la subclase `QuoteApp` que tienes crear y configurarla como clase de la aplicación en el `manifest.xml`.

Utiliza este código dentro del DSL del módulo para definir la dependencia de la base de datos:

```
single {
    androidContext().getInMemoryDatabase().also { database ->
        CoroutineScope(Dispatchers.IO).launch {
            database.quoteDao()
                .insertQuotes(QuoteProvider.programmingQuotes.map {
                    it.toEntity()
                })
        }
    }
}
```

Este código además de crear la base de datos, carga en la tabla quotes de la base de datos las citas que hay definidas en el objeto QuoteProvider.

Entrega

Comprime el proyecto en un zip y súbelo a aules.

Menu→File→Export→Export to zip file...